UXL SKILLS EVALUATOR

Add a Specialized Evaluation Target

From a new machine, device, or laboratory lane to trusted UXL evidence

Engineering guide · qualification, dispatch, evidence, and promotion

UXL
Unified Acceleration Foundation

A target adapter has five parts — and one shared evidence contract

Target integration changes; the evidence contract does not.

Labels route the job. Real probes, a passing oracle, and complete artifacts qualify the lane.

1. Write the capability contract before touching the runner

- **Name the missing capability** — Device, backend, topology, driver, instruction set, or other property hosted runners cannot reproduce faithfully.
- **Declare the task contract** — Failure, workflow stages, hardware class, environment, verifier, reward floor, and expected artifacts.
- **Set the trust boundary** — Who may dispatch reviewed work, which repository is allowed, and whether any credentials are permitted.
- **Define the evidence contract** — Actual device probes, software and container provenance, complete Harbor job data, verifier output, and task artifacts.
- **Choose the first exit criterion** — A deterministic oracle must earn reward 1.0 on the real target before model tokens are spent.

2. Add four public, reviewable pieces

- **Qualification task** — Create evaluation/harbor/tasks/<target-oracle>/ with instruction, pinned environment, oracle solution, and deterministic tests.
- **Target adapter — Configure target-adapter.json; the shared run_target_adapter.py verifies the SHA, probes the host, runs Harbor, and enforces the oracle gate.**
- **Provenance probe** — Record non-secret OS, runner, device, driver/runtime, container, toolchain, and Git revision evidence; add platform-specific probes when the shared collector is insufficient.
- **Portfolio and operator contract** — Declare hardware/environment/workflow metadata in suites.json and document labels, device mapping, limitations, and artifact handling.

3. Make the oracle prove the real execution path

- **Verify the immutable checkout** — Require a full 40-character evaluator SHA and compare it with git rev-parse HEAD.
- **Probe the host** — Confirm architecture, device interface, driver/runtime visibility, permissions, Docker, Python, and the harness version.
- **Probe the pinned container** — Map the actual device and required read-only runtime libraries; enumerate, compile, and run a minimal target workload.
- **Run one Harbor oracle** — Use the declared qualification task, one concurrent trial, and the oracle agent.
- **Fail closed** — Require the expected trial count and reward floor; always retain logs and artifacts when any gate fails.

4. Keep dispatch in a small private control repository

- **Manual input only** — workflow_dispatch accepts a reviewed 40-character source commit; no pull_request, pull_request_target, or public repository_dispatch trigger.
- **Exact capability labels** — runs-on includes self-hosted, OS/architecture, target capability, platform, and trust-tier labels.
- **Immutable public implementation** — The workflow checks out the public skills repository at the supplied SHA with clean checkout and persisted credentials disabled.
- **Pinned workflow dependencies** — Pin checkout and artifact actions to reviewed full commit SHAs and update them through review.
- **Complete failure evidence** — Upload the entire Harbor job directory with if: always(); restrict the runner or runner group to the control repository.

5. Prepare the machine, then register one ephemeral job

- **Install the target stack** — OS, driver/runtime, Docker, Python 3.12, device permissions, and the current GitHub runner offered by the control repository.
- **Use a dedicated identity and workspace** — Grant only required device groups and keep the runner work directory separate from personal files.
- **Register with --ephemeral** — Use capability and trust labels; for the first rollout, start one waiting job instead of installing a permanent service.
- **Keep the network one-way** — The runner needs outbound HTTPS to GitHub; no inbound firewall port is required.
- **Start or resume safely — Windows/WSL uses start-ephemeral-wsl-runner.ps1, which resumes a matching offline registration after reboot and refuses ambiguous state; native Linux may run the ephemeral agent directly.**

6. Dispatch a reviewed commit and require the oracle gate

- **1 — Review** — Select an immutable public evaluator commit and review the adapter and qualification task at that revision.
- **2 — Attach** — Start the ephemeral runner and confirm GitHub reports the expected name and labels online.
- **3 — Dispatch** — Run the private workflow with the exact commit SHA; concurrency stays one for qualification.
- **4 — Gate** — Require the real device probe, container probe, Harbor result, expected trial count, and reward 1.0.
- **5 — Preserve** — Download the always-uploaded job artifact before inspecting or cleaning the dedicated runner workspace.

The operator runs one short, reviewable sequence

- **Record the source revision** — git rev-parse HEAD — use the reviewed 40-character SHA, never a moving branch name.
- **Attach a Windows/WSL runner** — .\scripts\runner\start-ephemeral-wsl-runner.ps1 -Repository owner/private-runner-control -Labels 'uxl,<project>,<device>,windows-wsl,<trust-tier>'
- **Dispatch the private workflow** — gh workflow run <platform-oracle.yml> -R owner/private-runner-control -f skills_commit=<40-character-sha>
- **Watch the result** — gh run watch <run-id> -R owner/private-runner-control --exit-status
- **Preserve the evidence — Download the complete artifact ZIP before inspecting or cleaning the runner workspace.**
- **Stage it — python scripts/import_harbor_artifact.py <artifact.zip>; review the sanitized qualification-record.json before publication.**

7. Import the result and inspect it like any hosted run

- **Download — Keep the complete artifact ZIP intact; it carries provenance, results, trajectories, verifier output, configs, and probes.**
- **Stage — Run python scripts/import_harbor_artifact.py <artifact.zip>; the importer checks hashes and stages a sanitized candidate.**
- **Review — Check the public labels and limitations, then publish only qualification-record.json through normal review.**
- **Audit — Inspect the task, verifier, trajectory, artifacts, configs, and provenance before accepting the lane.**
- **Classify failures — Provisioning, network, driver, container, and runner failures are infrastructure failures, not skill failures.**

8. Add model trials only after the oracle and task headroom exist

- **Keep qualification credential-free** — Do not add model credentials until the environment and oracle path have passed review.
- **Use a task that needs the target** — A maintainer-backed failure must genuinely depend on the declared device or topology and leave room for the model to fail.
- **Match all three arms** — No skill, previous skill, and candidate skill use the same host, task revision, image, agent/harness, model, reasoning effort, attempts, timeouts, and concurrency.
- **Use enough trials** — Three attempts per arm for calibration; five for promotion evidence.
- **Quality remains the gate** — Compare tokens, cost, and runtime only after verified success; never trade lower success for cheaper runs.

Definition of done: a qualified lane, not a trusted black box

- **Immutable and isolated** — Only the private manual dispatcher can send a reviewed public revision to an ephemeral runner.
- **Actually qualified** — Host and pinned-container probes identify the intended device, and the real oracle earns reward 1.0.
- **Fully attributable** — The artifact records task, skill, model when applicable, harness, toolchain, software, hardware, Git, attempts, and failures.
- **Reviewable anywhere — Private logs stay access-controlled; the sanitized qualification record uses the shared dashboard and schema.**
- **Owned and maintainable — A named lane owner keeps target-adapter.json, labels, probes, images, runtime guidance, and limitations current.**
- **Honest scope** — The lane proves only its declared capability and revisions; it is not a vendor endorsement or universal compatibility claim.